

Freight Rate Prediction

Machine Learning Engineer Assessment — Spotter

Miguelangel Mosquera Martín · migelangeles14@gmail.com

1. Objective

Predict per-load freight rates for 12,000 November–December 2025 loads (**validation.csv**) and for a fixed Lexington→Fort Wayne lane across all 31 days of December 2025 (**december_chart_inputs.csv**), trained on 48,000 labeled historical loads from January–October 2025 (**train_test.csv**).

2. Data Quality Review

Before modeling, the training data was audited for consistency. Three issues were found and addressed explicitly rather than silently:

- **Missing values:** weight was missing in 300 of 48,000 training rows and market_index in 374 rows (165 and 249 respectively in the validation set). Both are numeric columns, so each missing cell was replaced with the **median** of that column, computed only from the training fold to avoid leakage.
- **Invalid negative weights:** 292 training rows (145 validation rows) had a negative weight, which is not physically possible. Rather than guess a sign-flip correction, these were treated as missing data and resolved with the same median imputation as above.
- **Unseen cities in validation:** eight pickup/delivery cities in validation.csv (Chicago, Norfolk, San Diego, Charlotte, Knoxville, Allentown, Laredo, Jackson) never occur in train_test.csv. One-hot encoding city names would give the model no information for these rows. Instead, the main model uses continuous pickup_lat/lon and delivery_lat/lon features, which let it generalize geographically to a city it has never seen.

3. Validation Strategy

Because the data is time-stamped and the task is explicitly to forecast a future period (November–December) from a past one (January–October), the training data was split **chronologically** rather than randomly: rows before October 1, 2025 were used for training, and the full month of October was held out for evaluation. A random or k-fold split would let the model see rows from the same week it is being tested on, inflating the validation score in a way that would not hold up on genuinely future data. The chronological holdout mirrors the real prediction task.

4. Modeling Approach

december_chart_inputs.csv does not include market_index, quote_signal, or coordinates — only distance, weight, equipment, city names and date. Rather than fabricate values for the missing columns, two separate models were trained on the same chronological split:

- **Full model** — every feature available in validation.csv (distance, weight, coordinates, equipment, date, market index, quote signal) — used to score the 12,000 validation loads.
- **Lane model** — only the features available in december_chart_inputs.csv (distance, weight, equipment, pickup/delivery city, date) — used for the December chart.

Three candidate algorithms — Linear Regression, Random Forest, and Gradient Boosting (all from scikit-learn) — were trained for each feature set and evaluated on the October holdout using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 . The model with the lowest holdout MAE was selected for each use case (highlighted below), then refit on the full training set before generating predictions.

Full model — used for validation_predictions.csv

Model	MAE (\$)	RMSE (\$)	R^2
Linear Regression	141.74	652.13	0.818
Random Forest	287.50	780.14	0.740
Gradient Boosting	159.37	659.73	0.814

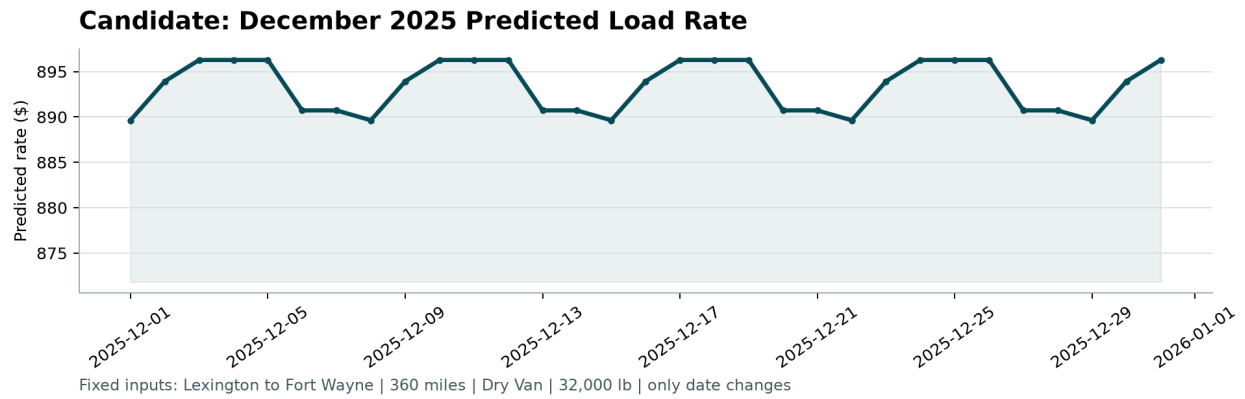
Lane model — used for december_predictions.csv

Model	MAE (\$)	RMSE (\$)	R^2
Linear Regression	172.61	655.22	0.816
Random Forest	218.74	752.76	0.757
Gradient Boosting	154.00	655.94	0.816

Linear Regression achieved the lowest error for the full model ($R^2 = 0.82$): the underlying relationship is dominated by an almost-linear effect of distance on rate, which favors a linear model over tree ensembles. Gradient Boosting performed best for the lane model, where the remaining signal is a subtler day-of-week pattern rather than one dominant continuous variable. Random Forest underperformed both alternatives in both cases.

5. December 2025 Rate Forecast

Generated by the provided score.py from the completed december_predictions.csv. Pickup, delivery, distance, equipment and weight are fixed; only the date changes across the 31 rows.



The lane model recovers a weekly cycle — higher mid-week, lower on weekends — consistent with the day-of-week effect observed in the historical rate-per-mile data for this route, without ever being told to look for one.

6. Reproducing These Results

Code, dependencies and run instructions are in the accompanying GitHub repository. In short:

```
python -m venv .venv && source .venv/bin/activate
python -m pip install -r requirements.txt
python src/train.py
python src/predict.py
python score.py --predictions validation_predictions.csv --december-predictions december_predictions.csv
```